EthioNikah — Islamic Marriage Platform

Technical & Security Documentation

Version: 1.0
Date: August 2026
Prepared for: Client Handover
Domain: https://ethionikah.com

Table of Contents

1. System Overview
2. Technology Stack
3. Architecture
4. Security Architecture
5. Authentication System
6. Two-Factor Authentication (2FA)
7. Account Lockout & Brute Force Protection
8. Rate Limiting
9. Admin Security & Access Control
10. Data Protection & Privacy
11. Audit Logging
12. Real-Time Communication
13. File Upload & Media Security
14. Frontend Security
15. Infrastructure & Deployment
16. Performance & Low-Bandwidth Optimization
17. Feature Modules
18. Database Schema
19. API Reference
20. Maintenance & Operations

---

1. System Overview

EthioNikah is a Sharia-compliant Islamic marriage platform designed for the Ethiopian Muslim community. It provides a secure, verified, and moderated environment for individuals seeking marriage in accordance with Islamic principles.

Core Principles

- **Sharia Compliance:** Wali (guardian) involvement for female users, Islamic tradition matching, halal communication guidelines
- **Security First:** Multi-layered authentication, encryption, rate limiting, and audit logging
- **Privacy by Design:** Health information encrypted, granular sharing controls, soft-delete with data retention windows
- **Low-Bandwidth Optimized:** Designed to work on slow 2G/3G connections common in Ethiopia
- **Multi-Language:** Full internationalization supporting English, Amharic, Arabic, and Oromo

---

2. Technology Stack

Backend

Component	Technology
Runtime	Node.js (v22+) with Express.js
Database	PostgreSQL with Prisma ORM v6
Cache	Redis (auth caching, rate limiting, sessions)
Real-Time	Socket.io with Redis adapter (multi-process)
Authentication	JWT (access + refresh tokens), bcrypt
2FA	Speakeasy (TOTP), QR code generation
Validation	Zod schema validation
File Storage	Cloudflare R2 (production) / Local filesystem (fallback)
Process Manager	PM2 (cluster mode)
Reverse Proxy	Nginx with HTTPS (Let's Encrypt)

Frontend

Component	Technology
Framework	Next.js 16 (App Router) with React
Styling	Tailwind CSS with custom Islamic-themed design
State Management	Zustand (messages), React Context (auth, theme, i18n)
Animations	Framer Motion
Icons	Lucide React
HTTP Client	Axios with interceptors and caching
Internationalization	Custom i18n (en, am, ar, om)
PWA	Installable, manifest, offline indicators

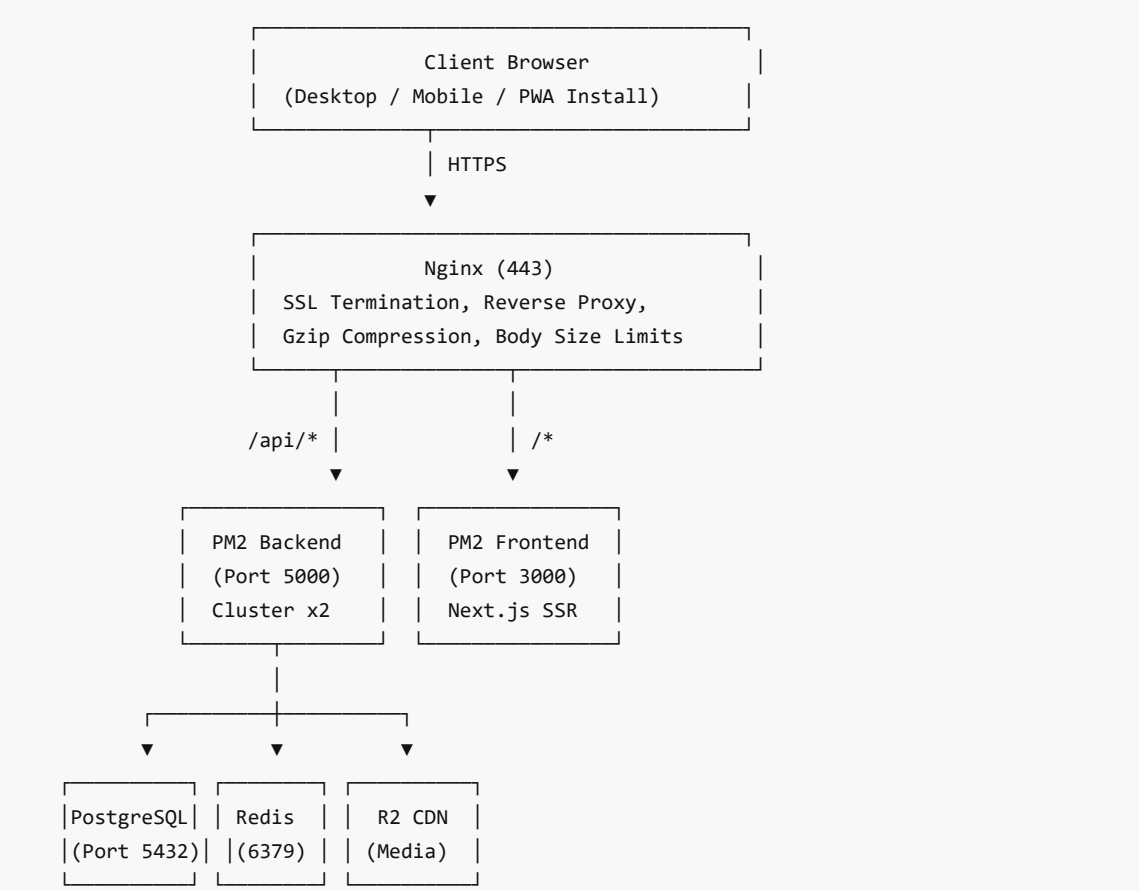
Infrastructure

Component	Technology
VPS	Linux server
Web Server	Nginx (reverse proxy, SSL termination)
SSL/TLS	Let's Encrypt (auto-renewal)
Process Management	PM2 (cluster mode for backend)

Database	PostgreSQL
Cache	Redis
CDN/Storage	Cloudflare R2 for media files

---

3. Architecture



Backend Structure

```
backend/src/
├── config/          # Database, Redis, Auth, R2, Storage
├── controllers/     # Business logic (15+ controllers)
├── middleware/      # Auth, rate limiting, validation, upload security
├── routes/          # Express route definitions
├── validators/      # Zod input validation schemas
├── utils/           # Email, encryption, audit logging, notifications
├── sockets/         # Socket.io real-time handlers
└── server.js        # Application entry point
```

Frontend Structure

```
frontend/src/
├── app/              # Next.js App Router pages
│   ├── admin/        # Admin panel pages
│   ├── (auth)/       # Login, register, forgot-password
│   └── (user)/       # Profile, matches, messages, settings
├── components/       # Reusable UI components
├── context/          # React Context providers
├── services/         # API service layer
├── hooks/            # Custom React hooks
├── lib/              # Utilities, API client
├── types/            # TypeScript definitions
└── i18n/             # Translations (en, am, ar, om)
```

---

4. Security Architecture

Security Layers (Defense in Depth)

```
Layer 1: Network Security
  ├── HTTPS/TLS (Let's Encrypt)
  ├── Nginx reverse proxy (hides backend)
  ├── Security headers (X-Frame-Options, X-Content-Type-Options, etc.)
  └── Unlimited body size for uploads (configured per route)

Layer 2: Application Security
  ├── JWT authentication (access + refresh tokens)
  ├── Role-based access control (RBAC)
  ├── Input validation (Zod schemas on every endpoint)
  ├── Rate limiting (per-endpoint, Redis-backed)
  └── CORS configuration

Layer 3: Authentication Security
  ├── bcrypt password hashing (10 rounds)
  ├── Progressive account lockout (brute force protection)
  ├── Two-factor authentication (TOTP + backup codes)
  ├── Trusted device tokens (skip 2FA on trusted devices)
  └── Session invalidation via token versioning

Layer 4: Admin Security
  ├── Super Admin role (exempt from 2FA, full access)
  ├── Admin 2FA enforcement (requireAdmin2FA middleware)
  ├── Permission-based access control (granular permissions)
  ├── Admin 2FA gate (frontend forced setup)
  └── Security block (auto-block on cross-role access attempts)

Layer 5: Data Security
  ├── Health info encryption (AES-256)
  ├── Soft-delete with 30-day restoration window
  ├── Audit logging (all admin actions)
  └── Password-protected account deletion
```

```
Layer 6: Real-Time Security
  ├── Socket.io authentication (JWT verified on connection)
  ├── Message rate limiting
  ├── Upload concurrency limiting
  └── Connection management per user
```

---

5. Authentication System

JWT Token Architecture

The system uses a dual-token authentication strategy:

Token Type	Expiry	Purpose
Access Token	15 minutes	API authentication, included in `Authorization: Bearer` header
Refresh Token	7 days	Seamless token renewal without re-login
Temp Token	5 minutes	2FA verification step (short-lived)

Token Generation

```
// Access token contains: userId, role, adminRole, tokenVersion
accessToken = jwt.sign({ userId, role, adminRole, tokenVersion }, JWT_SECRET, { expiresIn:
"15m" });

// Refresh token contains: userId, tokenVersion
refreshToken = jwt.sign({ userId, tokenVersion }, JWT_REFRESH_SECRET, { expiresIn: "7d" });
```

Session Invalidation (Token Versioning)

Every user has a `tokenVersion` field in the database. When a session needs to be invalidated (password change, 2FA enable/disable, account suspension), the token version is incremented. The auth middleware compares the JWT's token version with the database version — any mismatch immediately rejects the request.

Auth Middleware Flow

```
Request → Extract Bearer Token → Verify JWT Signature
        → Check Redis Cache (30s TTL)
          ├── Cache Hit → Verify tokenVersion → Allow/Deny
          └── Cache Miss → Query Database
             → Check account exists, not deleted, not suspended
             → Verify tokenVersion matches
             → Cache result in Redis
             → Allow/Deny
```

Password Security

- **Hashing:** bcrypt with 10 rounds
- **Storage:** Only `passwordHash` stored — plaintext never logged or transmitted
- **Reset:** Time-limited reset tokens with rate limiting

- **Change:** Requires current password verification

JWT Secret Validation

The server refuses to start in production mode if `JWT_SECRET` or `JWT_REFRESH_SECRET` are not set or are shorter than 32 characters.

---

6. Two-Factor Authentication (2FA)

Overview

The platform implements TOTP-based 2FA using the Speakeasy library, compatible with Google Authenticator, Microsoft Authenticator, and Authy.

2FA Flow

```
User Login
    │
    ├── Password Valid?
    │   ├── No → Record failed attempt → Return 401
    │   └── Yes ↓
    │
    ├── 2FA Enabled?
    │   ├── No → Issue access + refresh tokens → Login complete
    │   └── Yes ↓
    │       ├── Trusted Device?
    │       │   ├── Yes → Skip 2FA → Issue tokens → Login complete
    │       │   └── No → Issue temp token (5min) → Return requiresTwoFactor: true
    │       │           │
    │       │           └── User enters 6-digit code
    │       │               ├── Valid → Issue tokens → Login complete
    │       │               └── Invalid → Record failed attempt
    │       │                   ├── 3 fails → 6-hour lockout
    │       │                   └── 6 fails → Permanent lock (contact admin)
    │       │
    │       └── Backup code option (10 single-use codes)
```

2FA Features

- **QR Code Setup:** Generated via `qrcode` library, scanned by authenticator app
- **Backup Codes:** 10 single-use codes (bcrypt-hashed, tracked as used)
- **Trusted Devices:** Device token stored in localStorage, skips 2FA for 30 days
- **2FA Lockout:** 3 failed attempts → 6-hour lock; 6 total → permanent lock
- **Email Alerts:** Sent on 2FA enable, disable, and reset events
- **Admin Reset:** Super admin can reset 2FA for locked-out admins

2FA for Admins

Role	2FA Required	Enforcement
Super Admin	No (exempt)	Can log in without 2FA
Admin	Yes	`requireAdmin2FA` middleware on all admin API routes

Admin (new, no 2FA)	Prompted	Frontend `Admin2FAGate` forces 2FA setup after login
Regular User	Optional	Can enable in settings

---

7. Account Lockout & Brute Force Protection

Progressive Lockout System

The platform implements an escalating lockout mechanism to prevent brute force attacks:

Round	Failed Attempts	Lockout Duration
1	3 attempts	5 minutes
2	3 attempts	30 minutes
3	3 attempts	12 hours
4	3 attempts	3 days
5	3 attempts	**Permanent** (contact admin)

Additional Protection

- **15 total failed attempts** in any 10-day window → automatic permanent lock + account flagged for admin review
- Lockout state persists across sessions and devices
- Locked users see a countdown timer and "Contact Admin" link (shown from round 3+)
- Admins can manually unlock accounts

2FA-Specific Lockout

Failed 2FA Attempts	Consequence
3	6-hour temporary lock
6	Permanent lock (requires admin reset)

---

8. Rate Limiting

Rate limiting is implemented using `express-rate-limit` with Redis as the distributed store (falls back to in-memory if Redis is unavailable).

Rate Limit Policies

Endpoint Type	Limit	Window	Notes
General API	100 requests	15 min	Per IP
Login	5 attempts	15 min	Per IP, triggers IP ban on exceed
Login IP Ban	1 ban	1 hour	Auto-ban after login limit exceeded

Registration	3 accounts	1 hour	Per IP
Password Reset	3 requests	15 min	Per IP
Message Sending	30 messages	1 min	Per user
Message Upload	10 uploads	1 min	Per user
Profile Update	5 updates	15 min	Per user
Photo Upload	10 photos	15 min	Per user
Match Actions	30 actions	1 min	Per user (like, pass, favorite)
Contact Support	3 requests	1 hour	Per IP
Authenticated API	200 requests	1 min	Per user

Rate Limit Headers

All rate-limited responses include standard headers:

- `RateLimit-Limit` : Maximum requests allowed
- `RateLimit-Remaining` : Requests remaining in window
- `RateLimit-Reset` : Seconds until window resets
- `Retry-After` : Seconds to wait before retrying (on 429)

---

9. Admin Security & Access Control

Role Hierarchy

```
Super Admin
  ├── Full system access
  ├── Exempt from 2FA requirement
  ├── Can create/delete admins
  ├── Can reset admin 2FA
  └── Cannot be modified by other admins

Admin
  ├── Permission-based access (granular)
  ├── 2FA required (enforced at API + frontend level)
  ├── Cannot access super admin functions
  └── Can be suspended/deleted by super admin

Regular User
  ├── Access to user-facing features only
  ├── Cannot access any admin routes
  └── Cross-role access attempts trigger security block
```

Permission System

Admins have granular permissions stored in the `AdminPermission` table. Available permissions include:

Category	Permissions
Users	`users.view`, `users.edit`, `users.delete`, `users.restore`
Verifications	`verifications.view`, `verifications.approve`, `verifications.reject`
Reports	`reports.view`, `reports.resolve`
Complaints	`complaints.view`, `complaints.resolve`
Subscriptions	`subscriptions.view`, `subscriptions.manage`
Analytics	`analytics.view`
Matchmaking	`matchmaking.view`, `matchmaking.edit`
AI Features	`ai.fraud`, `ai.matchmaking`, `ai.moderation`, `ai.recommendations`
Settings	`settings.view`, `settings.edit`, `settings.feature_toggles`
Backups	`backup.view`, `backup.create`, `backup.restore`, `backup.delete`
Books	`books.view`, `books.manage`
Notifications	`notifications.templates`, `notifications.send`

Admin 2FA Enforcement

Backend (API Level):

- `requireAdmin2FA` middleware applied to all admin route files
- Checks `twoFactorEnabled` in database for non-super-admin admins
- Returns `403 ADMIN_2FA_REQUIRED` if 2FA is not enabled
- Super admins bypass this check entirely

Frontend (UI Level):

- `Admin2FAGate` component wraps all admin panel pages
- Detects non-super-admin admins without 2FA
- Forces 2FA setup screen (cannot access admin panel until 2FA is configured)
- Provides QR code setup and verification in a modal overlay

Security Block (Cross-Role Protection)

If a regular user attempts to access an admin route, or an admin attempts to access a user-only route:

1. A `security-block` API call is made
2. The user's account is flagged for review
3. The user is immediately logged out
4. Redirected to the login page

---

10. Data Protection & Privacy

Health Information Encryption

All health-related data (HIV status, STD status, hereditary diseases, chronic conditions, disabilities) is encrypted at rest using AES-256 encryption before being stored in the database.

```
const { encrypt, decrypt } = require("../utils/encryption");
// Health fields are encrypted before database insertion
hivStatus: encrypt(hivStatus),
stdDetails: encrypt(stdDetails),
```

Data Sharing Controls

Users have granular control over what information is shared with potential matches:

- `shareWithMatches` flag for health information
- Match preferences for health conditions (e.g., `hivMatchPreference`, `stdMatchPreference`)
- Profile visibility settings

Account Deletion

- **Soft Delete:** Account is marked as deleted but data retained for 30 days
- **Restoration:** Users can restore their account within 30 days
- **Permanent Deletion:** After 30 days, data is permanently purged
- **Verification:** Permanent deletion requires OTP verification (married users only)

Sensitive Data Handling

Data Type	Protection
Passwords	bcrypt hash (10 rounds), never logged
Health Info	AES-256 encryption at rest
2FA Secrets	Stored in database (not accessible via API)
Backup Codes	bcrypt-hashed, tracked as used
JWT Secrets	Environment variables, min 32 chars in production
API Tokens	Stored in localStorage with httpOnly cookie backup

---

11. Audit Logging

Overview

All admin actions are logged in the `AuditLog` table with the following fields:

Field	Description
`id`	Unique log entry ID
`actorId`	Admin user ID who performed the action
`actorType`	Always "ADMIN" (extensible for future roles)
`action`	Action performed (e.g., `user_suspend`, `verification_approve`)

`resource`	Resource type (e.g., `user`, `verification`, `complaint`)
`resourceId`	ID of the affected resource
`details`	JSON object with action-specific details
`ipAddress`	IP address of the admin
`createdAt`	Timestamp

Logged Actions

- User suspension/activation
- Verification approval/rejection
- Complaint resolution
- Report handling
- Admin creation/deletion
- Permission changes
- 2FA resets
- Payment management
- Subscription management
- Feature toggle changes
- Backup creation/restoration

Audit Log Access

- Super admins can view all audit logs
- Admins can view their own activity history
- Logs are displayed in the admin panel with filtering by action, resource, and admin
- Admin activity stats (30-day rolling window) shown on the admin management page

---

12. Real-Time Communication

Socket.io Architecture

The platform uses Socket.io with a Redis adapter for multi-process real-time communication:

```
Client Browser ↔ Nginx ↔ PM2 Backend (Cluster x2)
                                │
                                └── Redis Adapter (shared state)
```

Real-Time Features

Feature	Description
Instant Messaging	Real-time text, image, and file messages
Typing Indicators	Live typing status in conversations
Read Receipts	Message read status updates
Online Status	User presence indicators
Notifications	Real-time notification delivery

Verification Status	Live verification status updates
Payment Status	Real-time payment confirmation

Socket Security

- JWT authentication required on socket connection
- User identity verified before joining private rooms
- Message rate limiting enforced
- Upload concurrency limiting (prevents flooding)

---

13. File Upload & Media Security

Upload Pipeline

```
Client → Multipart Form → Multer (memory storage) → File Validation
  → Upload Security Middleware → R2 Cloudflare (production) or Local (fallback)
```

Upload Security

- **File Type Validation:** Only allowed MIME types accepted
- **File Size Limits:** Enforced per route (Nginx + Express)
- **Upload Security Middleware:** Verifies file integrity before processing
- **Image Blur Detection:** Client-side blur detection for verification photos (Laplacian variance)
- **R2 Storage:** Production uses Cloudflare R2 for CDN-backed media delivery
- **Local Fallback:** If R2 is unavailable, files are stored locally with URL paths

Supported File Types

Purpose	Types	Notes
Profile Photos	JPEG, PNG, WebP	Blur detection on upload
Verification Photos	JPEG, PNG	ID + selfie with blur check
Message Images	JPEG, PNG, WebP, GIF	Rate limited
Message Files	PDF, DOC, DOCX	Size limited
Marriage Success Photos	JPEG, PNG	Admin moderated

---

14. Frontend Security

Route Protection

The `RouteGuard` component provides client-side route protection:

Route Type	Protection
Public Routes	`/`, `/login`, `/register`, `/forgot-password`, `/verify-email`, `/verify-phone`
User Routes	Requires authentication + active account

Admin Routes	Requires admin role + 2FA (if non-super-admin)
User-Only Routes	Admins blocked (triggers security block)

API Client Security

- **Auth Token Injection:** Automatically adds `Authorization: Bearer` header
- **Token Refresh:** Auto-refreshes expired access tokens using refresh token
- **Session Invalidation:** Detects `Session invalidated` error and redirects to login
- **Client-Side Caching:** GET responses cached for 15 seconds with stale-while-revalidate
- **Network Error Handling:** Serves cached data on network failure (offline resilience)
- **Timeout Protection:** 30s default, 3min for file uploads

Security Headers (Nginx)

```
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
```

Static Asset Caching

```
/_next/static/*  → Cache-Control: public, max-age=31536000, immutable
/application.png → Cache-Control: public, max-age=31536000, immutable
/manifest.webmanifest → Cache-Control: public, max-age=86400
```

---

15. Infrastructure & Deployment

Server Configuration

Component	Configuration
VPS IP	187.77.79.247
Project Path	`/var/www/ETHIONIKAH-/`
Nginx Config	`/etc/nginx/sites-enabled/ethionikah`
SSL	Let's Encrypt (auto-renewal)
Body Size	Unlimited (0) — no upload size restrictions at proxy level
Backend	PM2 cluster mode (2 instances, port 5000)
Frontend	PM2 fork mode (port 3000)
Redis	Running locally (no password, port 6379)
PostgreSQL	Running locally (port 5432)

Environment Variables

Backend (`.env`):

```
DATABASE_URL=postgresql://user:pass@localhost:5432/ethionikah?
connection_limit=5&pool_timeout=30
JWT_SECRET=<32+ character secret>
JWT_REFRESH_SECRET=<32+ character secret>
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
REDIS_URL=redis://localhost:6379
R2_ACCOUNT_ID=<Cloudflare R2 account ID>
R2_ACCESS_KEY_ID=<R2 access key>
R2_SECRET_ACCESS_KEY=<R2 secret key>
R2_BUCKET_NAME=<R2 bucket>
```

Frontend (`.env.local`):

```
NEXT_PUBLIC_API_URL=/api
```

> ***Important:*** *`NEXT_PUBLIC_*` variables are inlined at build time. The frontend must be rebuilt after changing these values.*

Deployment Process

```
# 1. Pull latest code
cd /var/www/ETHIONIKAH-
git pull origin main

# 2. If Prisma schema changed
cd backend
npx prisma generate
npx prisma db push    # NEVER use --force-reset

# 3. If frontend changed
cd ../frontend
npm run build

# 4. Restart services
pm2 restart ethionikah-backend ethionikah-frontend

# 5. Verify
pm2 status
pm2 logs ethionikah-backend --lines 10 --nostream
```

PM2 Process Management

Process Name	Mode	Instances	Port
`ethionikah-backend`	Cluster	2	5000
`ethionikah-frontend`	Fork	1	3000

Nginx Configuration

- Reverse proxies `/api/*` to backend (port 5000)
- Serves Next.js frontend (port 3000) for all other routes
- SSL termination with Let's Encrypt certificates
- Gzip compression enabled
- `client_max_body_size 0` (unlimited upload size)

---

16. Performance & Low-Bandwidth Optimization

The platform is specifically optimized for slow internet connections (2G/3G) common in Ethiopia.

Optimizations

Optimization	Description
API Timeouts	30s default, 3min for uploads — prevents premature failures
Client-Side Caching	GET responses cached 15s with stale-while-revalidate
Offline Resilience	Serves cached data when network fails
Static Asset Caching	JS/CSS bundles cached for 1 year (immutable)
Image Optimization	AVIF/WebP formats via Next.js Image
Gzip Compression	Enabled at Nginx level
Redis Auth Cache	30s TTL — avoids DB query on every request
Network Banner	Shows offline status banner when connection lost
Error Toast Duration	4s display for network errors (readable on slow devices)
PWA Support	Installable app, works with manifest

Network Status Detection

The `useNetworkStatus` hook monitors browser online/offline events and displays a red banner when the connection is lost, so users know the issue is network-related, not a platform error.

---

17. Feature Modules

User Features

Module	Description
Registration	Multi-step registration with profile, preferences, health info
Profile Management	Editable profile with photo upload, bio, preferences
Match System	AI-powered suggestions, like/pass, mutual matching
Messaging	Real-time chat with images, files, typing indicators
Verification	ID + selfie verification with blur detection

Favorites	Save and manage favorite profiles
Exclusive Partnership	Request exclusive partnership with a match
Wali System	Guardian involvement for female users
Notifications	Real-time + email notifications
Settings	Privacy, language, 2FA, account management
Membership	Premium plans (Gold, Platinum) with payment
Success Stories	Married users can share their success story
Books	Islamic marriage book purchases
Health Info	Encrypted health data with sharing controls

Admin Features

Module	Description
Dashboard	Overview stats, charts, recent activity
User Management	Search, filter, suspend, activate, delete users
Verifications	Review and approve/reject ID verifications
Reports	Handle user reports
Complaints	Resolve user complaints
Admin Management	Create/delete admins, manage permissions
Subscriptions	View and manage user subscriptions
Payments	Payment management and tracking
Analytics	Platform analytics and insights
Audit Logs	View all admin actions with filtering
Feature Toggles	Enable/disable platform features
Marriage Success	Moderate success story submissions
Books	Manage book catalog
Backups	Database backup management
Manual	Admin usage manual

---

18. Database Schema

Core Models

Model	Description	Key Fields
User	Platform users	email, phone, passwordHash, role, adminRole, twoFactorEnabled, tokenVersion
Profile	User profile data	nationality, occupation, bio, photos, religiousCommitment
SpousePreference	Match preferences	preferredAgeMin/Max, preferredMaritalStatus, waliInvolvement
HealthInfo	Encrypted health data	hivStatus (encrypted), stdStatus (encrypted), shareWithMatches
Wali	Guardian records	femaleUserId, name, relationship, phone, email
Verification	ID verification	userId, idPhotoUrl, selfiePhotoUrl, status, adminId
Match	User matches	senderId, receiverId, status (liked/passed/matched)
Message	Chat messages	senderId, receiverId, content, type, readAt
Notification	User notifications	userId, type, data, readAt
Subscription	Membership subscriptions	userId, plan, status, startDate, endDate
Payment	Payment records	userId, amount, status, method
AuditLog	Admin action logs	actorId, action, resource, resourceId, details
AdminPermission	Granular admin permissions	adminId, permission
Complaint	User complaints	userId, subject, status, adminId
Report	User reports	reporterId, reportedUserId, reason, status
BlockedUser	User blocks	blockerId, blockedId
Favorite	User favorites	userId, favoriteUserId
ProfileView	Profile view tracking	viewerId, viewedId
FeatureToggle	Platform feature flags	key, label, enabled, category
Boost	Profile boost	userId, expiresAt
Wallet	User wallet	userId, balance
UserContactCredit	Contact credits	userId, credits
ExclusivePartnership	Exclusive partnerships	userAId, userBId, status
PushSubscription	PWA push notifications	userId, endpoint, keys
MarriageSuccess	Success stories	userId, photos, story, status

Book	Islamic books	title, author, price, coverUrl
UserRestriction	Admin restrictions	userId, type, reason

---

19. API Reference

Authentication Routes (`/api/auth`)

Method	Endpoint	Auth	Description
POST	`/register`	None	Register new user
POST	`/login`	None	Login (email + password)
POST	`/login/phone`	None	Login (phone + password)
POST	`/refresh`	Refresh Token	Refresh access token
GET	`/me`	Access Token	Get current user details
POST	`/logout`	Access Token	Logout (invalidate session)
POST	`/forgot-password`	None	Request password reset
POST	`/reset-password`	None	Reset password with token
POST	`/security-block`	Access Token	Report security violation
POST	`/delete-account`	Access Token	Request account deletion
POST	`/delete-account/confirm`	Access Token	Confirm deletion with OTP
GET	`/2fa/status`	Access Token	Check 2FA status
POST	`/2fa/enable`	Access Token	Generate 2FA QR code
POST	`/2fa/confirm`	Access Token	Confirm 2FA setup
POST	`/2fa/disable`	Access Token	Disable 2FA (requires password + code)
POST	`/2fa/verify`	Temp Token	Verify 2FA code on login
POST	`/2fa/regenerate-codes`	Access Token	Generate new backup codes
GET	`/2fa/check-lockout`	Access Token	Check 2FA lockout status

Admin Routes (`/api/admin`)

Method	Endpoint	Auth	2FA	Description
GET	`/users`	Admin	Yes	List users with filters
GET	`/users/:id`	Admin	Yes	Get user details
POST	`/users/:id/suspend`	Admin	Yes	Suspend user

POST	`/users/:id/activate`	Admin	Yes	Activate user
POST	`/users/:id/pin`	Admin	Yes	Pin user for review
GET	`/badge-counts`	Admin	No	Dashboard badge counts
GET	`/activity`	Admin	Yes	Audit log activity
GET	`/audit-logs`	Admin	Yes	Full audit logs with filters
GET	`/analytics`	Admin	Yes	Platform analytics
GET	`/suspicious-activity`	Admin	Yes	Suspicious activity flags

Admin Management Routes (`/api/admin/admins`)

Method	Endpoint	Auth	Description
GET	`/`	Super Admin	List all admins
POST	`/create`	Super Admin	Create new admin
PUT	`/:id/permissions`	Super Admin	Update admin permissions
DELETE	`/:id`	Super Admin	Delete admin (requires password + 2FA)
POST	`/:id/reset-2fa`	Super Admin	Reset admin 2FA

User Routes

Method	Endpoint	Auth	Description
GET	`/api/profile`	User	Get own profile
PUT	`/api/profile`	User	Update profile
POST	`/api/profile/photo`	User	Upload profile photo
GET	`/api/matches/suggested`	User	Get suggested matches
POST	`/api/matches/like`	User	Like a profile
POST	`/api/matches/pass`	User	Pass on a profile
GET	`/api/messages/conversations`	User	List conversations
POST	`/api/messages/send`	User	Send message
POST	`/api/notifications`	User	List notifications
POST	`/api/verification/submit`	User	Submit verification photos
GET	`/api/verification/status`	User	Check verification status

---

20. Maintenance & Operations

Daily Operations

- **Auto Backup:** Database backup runs daily at midnight via PM2 scheduler
- **Conversation Expiry:** Pending conversation requests auto-expire (cron job)
- **Security Scan:** Server logs admins without 2FA on startup

Monitoring

```
# Check process status
pm2 status

# View backend logs (last 50 lines)
pm2 logs ethionikah-backend --lines 50 --nostream

# View backend errors only
pm2 logs ethionikah-backend --lines 50 --nostream --err

# Check Redis
redis-cli ping

# Check Nginx
nginx -t
systemctl status nginx

# Check PostgreSQL
systemctl status postgresql
```

Common Issues & Solutions

Issue	Cause	Solution
Network error on site	Database schema mismatch	Run `npx prisma db push` on VPS
Frontend API errors	`NEXT_PUBLIC_API_URL` not set	Ensure `.env.local` exists before `npm run build`
Browser shows old version	Next.js static caching	Hard refresh: `Ctrl+Shift+R`
Redis "client is closed"	Socket adapter lost connection	`pm2 restart ethionikah-backend`
Upload fails (413)	Nginx body size limit	Verify `client_max_body_size 0` in Nginx config
Admin can't log in	2FA not enabled (non-super-admin)	Super admin resets 2FA or admin enables it
AuditLog error	`adminId` vs `actorId` mismatch	All queries must use `actorId` (mapped to `admin_id` column)

Backup Strategy

- **Automated:** Daily database backup via PM2 scheduler
- **Manual:** Admin panel backup creation (super admin only)
- **Storage:** Backups stored locally on VPS

- **Restore:** Available via admin panel (super admin only)

Security Checklist

- [x] HTTPS enforced (Let's Encrypt)
- [x] JWT secrets ≥ 32 characters (validated on startup)
- [x] Password hashing (bcrypt, 10 rounds)
- [x] Rate limiting on all sensitive endpoints
- [x] Input validation on all API endpoints (Zod)
- [x] CORS configured
- [x] Security headers (Nginx)
- [x] 2FA for admins (enforced at API + UI level)
- [x] Progressive account lockout
- [x] Audit logging for all admin actions
- [x] Health data encryption at rest
- [x] Soft-delete with restoration window
- [x] Session invalidation via token versioning
- [x] File upload validation and security
- [x] Socket.io authentication
- [x] Cross-role access protection (security block)
- [x] PWA with offline indicators
- [x] Low-bandwidth optimization (timeouts, caching, compression)

Appendix A: Internationalization

The platform supports four languages:

Language	Code	Status
English	`en`	Complete
Amharic	`am`	Complete (audited, natural phrasing)
Arabic	`ar`	Complete
Oromo	`om`	Complete

Language is stored per-user in the `language` field and persists across sessions.

Appendix B: PWA Configuration

The platform is installable as a Progressive Web App:

- **Manifest:** `/manifest.webmanifest`
- **Icons:** Multiple sizes (192px, 512px)
- **Theme:** Islamic gold (#D4AF37) on dark background
- **Display:** Standalone mode
- **Install Prompt:** Shown on supported browsers

This documentation reflects the state of the EthioNikah platform as of August 2026. For questions or support, contact the development team.

© 2026 EthioNikah. All rights reserved.